universität freiburg

Machine Learning: II. Linear Methods

Dr. Simon Bultmann, Prof. Dr. Abhinav Valada

Winter Semester 2026-2027

October 26th, 2026

robot learning



Outline

1. Linear Regression
2. Classification & Logistic Regression
3. Conclusion



Outline

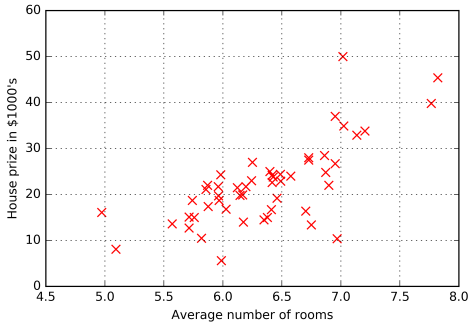
1. Linear Regression
2. Classification & Logistic Regression
3. Conclusion



Motivation: Predicting housing prices

- Let's say we only know the average number of rooms in an area
- Aim is to predict the prize for a house in that area

avg. # rooms	price
6.575	24
6.377	21.6
5.570	34.7
5.713	33.4
7.024	36.2
5.963	28.7
5.741	22.9
6.417	27.1
⋮	⋮



The Formal Regression Problem

Given a dataset of N instances, each instance consisting of:

- A feature vector $\boldsymbol{x}_i \in \mathbb{R}^D$ are D-dimensional vectors $\forall i \in \{1, \ldots, N\}$
- Target is $y_i \in \mathbb{R}$ uni-dimensional $\forall i \in \{1, \ldots, N\}$

Objective: Find a prediction model $\hat{y} := f : \mathbb{R}^D \mapsto \mathbb{R}$ that estimates y by minimizing a loss $J : \mathbb{R} \times \mathbb{R} \mapsto \mathbb{R}_+$

$$\arg\min_f \sum_{i=1}^{N} J(y_i,\, f(\boldsymbol{x}_i))\,.$$

Refined Objective: Find a parametric model $\hat{y} := f : \mathbb{R}^D \times W \mapsto \mathbb{R}$

$$\arg\min_{\boldsymbol{w} \in W} \sum_{i=1}^{N} J(y_i, f(\boldsymbol{x}_i; \boldsymbol{w}))\,.$$

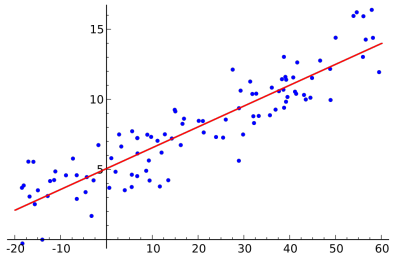
Linear Regression

The simplest regression model is the linear regression

$$f(\boldsymbol{x}_i;\ \boldsymbol{w}) = w_0 + w_1 x_{i,1} + ... + w_D x_{i,D} = w_0 + \boldsymbol{w}^\top \boldsymbol{x}_i,$$

where

- w_0 is a fixed offset, called the **bias**
- $\boldsymbol{w}$ is the weight vector or parameter vector



$$f(x;\ w_0 = 5.0;\ \boldsymbol{w} = [0.125]) = 5.0 + 0.125x$$

Linear Regression: A limited model?

Characteristics:

- Linear function of the weights / parameters $\boldsymbol{w} \in \mathbb{R}^{D+1}$
- Linear function of the input dimensions / variables $\boldsymbol{x} \in \mathbb{R}^{D+1}$

Question:

What **significant limitation** does this model have?

Nonlinear Mapping

- Can the function $f(x) = (x + 1)^2$ be approximated through a linear function?

Nonlinear Mapping

- Can the function $f(x) = (x+1)^2$ be approximated through a linear function?
- Yes, but only if we **map** the feature x into a new space:

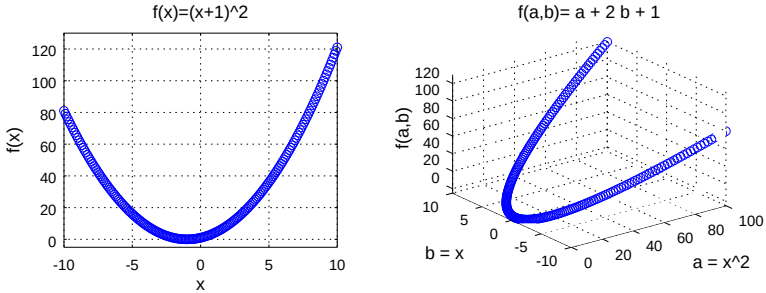


Figure 1: Mapping feature x into a new dimensionality $x \mapsto \phi(x) = (a, b)$

Linear Regression with Basis Functions

Nonlinear projection of features $\phi : \mathbb{R}^D \to \mathbb{R}^M$

Linear combinations of fixed **nonlinear functions** of the input variables:

$$f(\boldsymbol{x}_i, \boldsymbol{w}) = w_0 + \sum_{j=1}^{M} w_j \phi(\boldsymbol{x}_i)_j$$

where $\phi(\boldsymbol{x}_i)_j$ are known as **basis functions**.

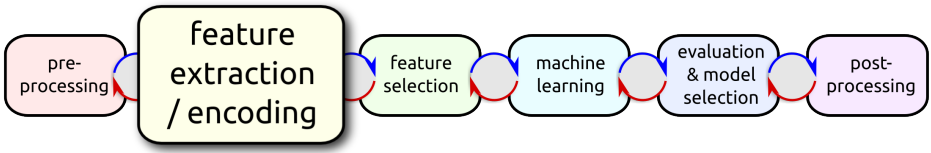
Characteristics of this **enhanced** linear regression model:

- $f(\boldsymbol{x}, \boldsymbol{w})$ is still a linear function of the weights / parameters w_i
- $f(\boldsymbol{x}, \boldsymbol{w})$ is a **nonlinear** function of the input dimensions / variables x_i

Linear Regression with Basis Functions (II)

Characteristics of this **enhanced** linear regression model:

- Adding basis functions may enlarge the dimensionality
- The use of fixed basis functions corresponds to an earlier step in the ML pipeline



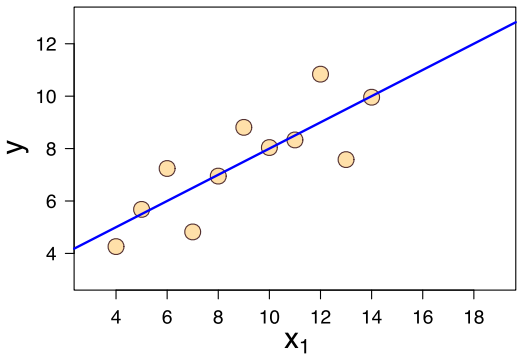
Combine features:

- We can use complex operations to combine features
- Combined features can be more expressive than their components
  (use expert knowledge!)

Interpretation of the Quality of the Model

Question:

How can we guess the quality of the regression model?



- Residual Error:

$$\epsilon_i = y_i - f(\boldsymbol{x}_i;\ \boldsymbol{w})$$

- Loss/Cost:

$$J(\boldsymbol{w}) = \sum_{i=1}^{N} J(y_i,\ f(\boldsymbol{x}_i;\ \boldsymbol{w}))$$

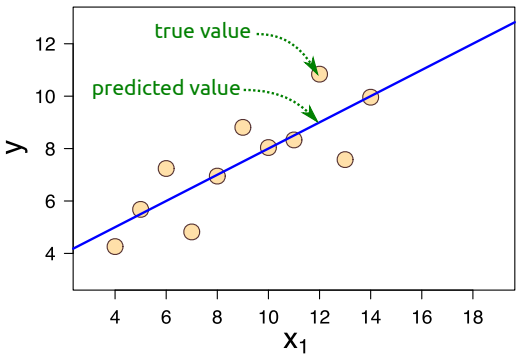
e.g.

$$J(y_i,\ f(\boldsymbol{x}_i;\ \boldsymbol{w})) = \epsilon_i^2$$

Interpretation of the Quality of the Model

Question:

How can we guess the quality of the regression model?



- Residual Error:

$$\epsilon_i = y_i - f(\boldsymbol{x}_i;\ \boldsymbol{w})$$

- Loss/Cost:

$$J(\boldsymbol{w}) = \sum_{i=1}^{N} J(y_i,\ f(\boldsymbol{x}_i;\ \boldsymbol{w}))$$

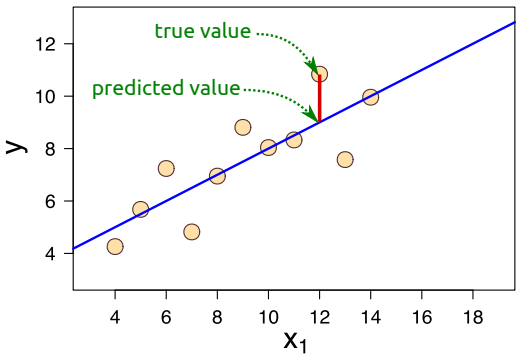
e.g.

$$J(y_i,\ f(\boldsymbol{x}_i;\ \boldsymbol{w})) = \epsilon_i^2$$

Interpretation of the Quality of the Model

Question:

How can we guess the quality of the regression model?



- Residual Error:

$$\epsilon_i = y_i - f(\boldsymbol{x}_i;\ \boldsymbol{w})$$

- Loss/Cost:

$$J(\boldsymbol{w}) = \sum_{i=1}^{N} J(y_i,\ f(\boldsymbol{x}_i;\ \boldsymbol{w}))$$

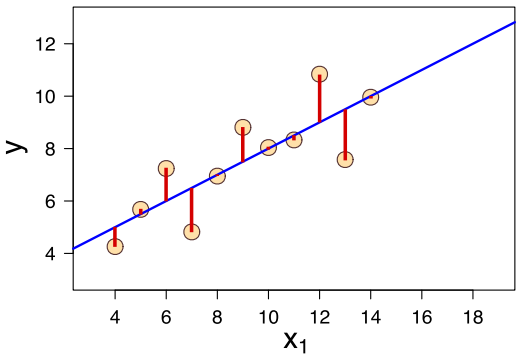
e.g.

$$J(y_i,\ f(\boldsymbol{x}_i;\ \boldsymbol{w})) = \epsilon_i^2$$

Interpretation of the Quality of the Model

Question:

How can we guess the quality of the regression model?



- Residual Error:

$$\epsilon_i = y_i - f(\boldsymbol{x}_i;\ \boldsymbol{w})$$

- Loss/Cost:

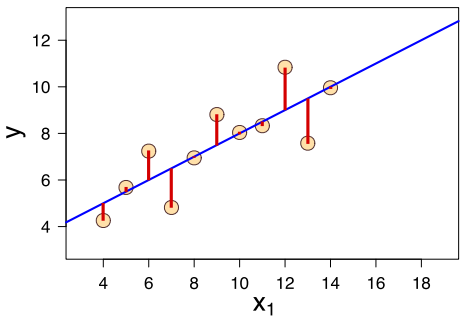
$$J(\boldsymbol{w}) = \sum_{i=1}^{N} J(y_i,\ f(\boldsymbol{x}_i;\ \boldsymbol{w}))$$

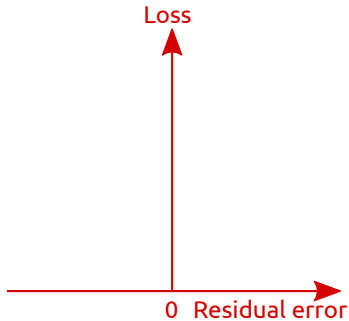
e.g.

$$J(y_i,\ f(\boldsymbol{x}_i;\ \boldsymbol{w})) = \epsilon_i^2$$

Alternative Loss Functions

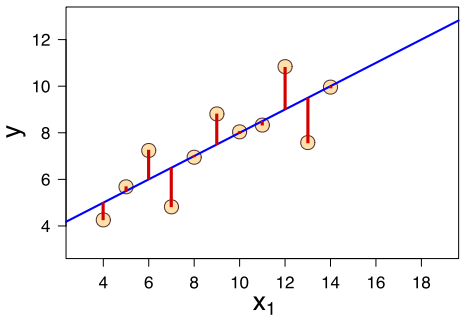
Typical loss choices, either L_2: $J(y, \hat{y}) = (y - \hat{y})^2$, or L_1: $J(y, \hat{y}) = |y - \hat{y}|$

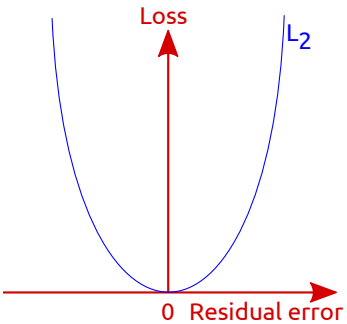




Alternative Loss Functions

Typical loss choices, either L_2: $J(y, \hat{y}) = (y - \hat{y})^2$, or L_1: $J(y, \hat{y}) = |y - \hat{y}|$



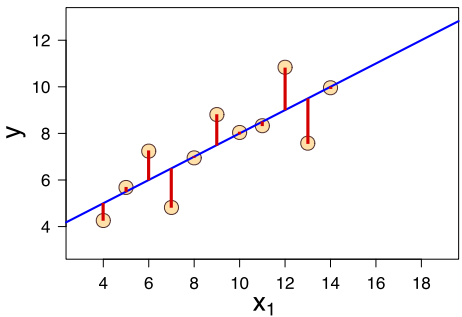


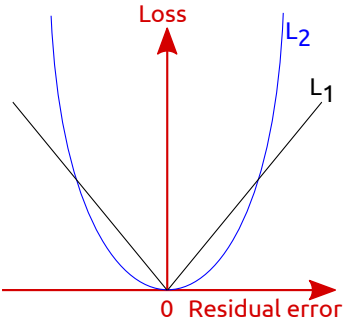
Discussion:

In which situations would you expect absolute loss (L_1) to be preferable compared to squared loss (L_2)?

Alternative Loss Functions

Typical loss choices, either L_2: $J(y, \hat{y}) = (y - \hat{y})^2$, or L_1: $J(y, \hat{y}) = |y - \hat{y}|$

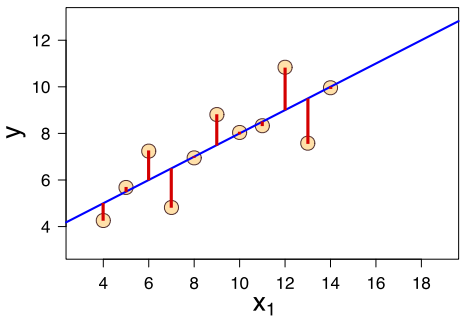


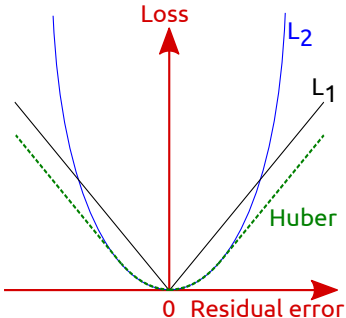


With L_2, **outliers** are heavily penalized and heavily influence the model. L_1 does not emphasize outliers as much.

Alternative Loss Functions

Typical loss choices, either L_2: $J(y, \hat{y}) = (y - \hat{y})^2$, or L_1: $J(y, \hat{y}) = |y - \hat{y}|$





The **Huber** loss tries to take the best from both approaches.

How Can We Derive the Weight Parameters w?

$$\arg\min_{\boldsymbol{w}} \sum_{i=1}^{N} J(y_i,\ f(\boldsymbol{x}_i;\ \boldsymbol{w}))$$

Weight vector $\boldsymbol{w}$ can be derived in (at least) two manners:

- **Option 1:** Determine the weights analytically.
- **Option 2:** Guess the entries of $\boldsymbol{w}$ and try to improve them iteratively

Assumptions Required for Analytical Solution

$$y_i = w_0 + w_1 x_1 + ... + w_D x_D + \varepsilon_i$$

The analytical solution is based on **three assumptions**. If violated, the model may fail or underperform.

- **A1:** The expected value of the residual errors is zero:

$$\forall i\colon \ \mathrm{E}(\varepsilon_i) = 0$$

- **A2:** The residual errors are uncorrelated and share the same variance:

$$\forall i\colon \ \mathrm{Var}(\varepsilon_i) = \sigma^2$$

- **A3:** The residual errors follow a normal distribution:

$$\varepsilon_i \sim \mathcal{N}(0,\, \sigma^2)$$

Optimization of w via Least Squares Loss Function

Assuming the slightly more elegant formulation for linear regression (in augmented vector notation):

$$\boldsymbol{y} = \boldsymbol{X}\boldsymbol{w} + \boldsymbol{\varepsilon}$$

we would like to minimize the squared error of the model:

$$\arg\min_{\boldsymbol{w}} \ \|\boldsymbol{\varepsilon}\|^2 = \arg\min_{\boldsymbol{w}} \ \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}\|^2$$

Optimization of w via Least Squares Loss Function

$$\arg\min_{\boldsymbol{w}} \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}\|^2 = \arg\min_{\boldsymbol{w}} (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w})^\top (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w})$$

$$\begin{aligned}
(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w})^\top (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}) &= \boldsymbol{y}^\top \boldsymbol{y} - (\boldsymbol{X}\boldsymbol{w})^\top \boldsymbol{y} - \boldsymbol{y}^\top \boldsymbol{X}\boldsymbol{w} + (\boldsymbol{X}\boldsymbol{w})^\top \boldsymbol{X}\boldsymbol{w} \\
&\rightarrow -(\boldsymbol{X}\boldsymbol{w})^\top \boldsymbol{y} - \boldsymbol{y}^\top \boldsymbol{X}\boldsymbol{w} + (\boldsymbol{X}\boldsymbol{w})^\top \boldsymbol{X}\boldsymbol{w} \\
&\rightarrow -\boldsymbol{w}^\top \boldsymbol{X}^\top \boldsymbol{y} - \boldsymbol{y}^\top \boldsymbol{X}\boldsymbol{w} + \boldsymbol{w}^\top \boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{w} \\
&\rightarrow -\boldsymbol{w}^\top \boldsymbol{X}^\top \boldsymbol{y} - \boldsymbol{y}^\top \boldsymbol{X}\boldsymbol{w} + \boldsymbol{w}^\top \boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{w} \\
&\rightarrow -2\boldsymbol{w}^\top \boldsymbol{X}^\top \boldsymbol{y} + \boldsymbol{w}^\top \boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{w}
\end{aligned}$$

Optimization of w via Least Squares Loss Function (II)

$$\arg\min_{\boldsymbol{w}} \; -2\boldsymbol{y}^\mathsf{T}\boldsymbol{X}\boldsymbol{w} + \boldsymbol{w}^\mathsf{T}\boldsymbol{X}^\mathsf{T}\boldsymbol{X}\boldsymbol{w}$$

Find optimum by setting the partial derivatives w.r.t. $\boldsymbol{w}$ to zero:

$$0 = \frac{\partial}{\partial \boldsymbol{w}} - 2\boldsymbol{y}^\mathsf{T}\boldsymbol{X}\boldsymbol{w} + \boldsymbol{w}^\mathsf{T}\boldsymbol{X}^\mathsf{T}\boldsymbol{X}\boldsymbol{w}$$

Use $\frac{\partial}{\partial \boldsymbol{w}} \boldsymbol{w}^\mathsf{T}\boldsymbol{A}\boldsymbol{w} = \boldsymbol{w}^\mathsf{T}(\boldsymbol{A} + \boldsymbol{A}^\mathsf{T})$ to derive:

$$0 = -2\boldsymbol{y}^\mathsf{T}\boldsymbol{X} + \boldsymbol{w}^\mathsf{T}(\boldsymbol{X}^\mathsf{T}\boldsymbol{X} + (\boldsymbol{X}^\mathsf{T}\boldsymbol{X})^\mathsf{T})$$

Optimization of w via Least Squares Loss Function (III)

$$
\begin{aligned}
0 = & -2\boldsymbol{y}^\top \boldsymbol{X} + \boldsymbol{w}^\top (\boldsymbol{X}^\top \boldsymbol{X} + (\boldsymbol{X}^\top \boldsymbol{X})^\top) \\
= & -2\boldsymbol{y}^\top \boldsymbol{X} + 2\boldsymbol{w}^\top \boldsymbol{X}^\top \boldsymbol{X} \\
\iff & \boldsymbol{y}^\top \boldsymbol{X} = (\boldsymbol{w}^\top \boldsymbol{X}^\top) \boldsymbol{X} \\
\iff & \boldsymbol{X}^\top \boldsymbol{y} = \boldsymbol{X}^\top \boldsymbol{X} \boldsymbol{w} \\
\iff & (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y} = \boldsymbol{w}
\end{aligned}
$$

Optimization of w

This delivers the analytical solution:

$$\boldsymbol{w} = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

Question: What is the costly part of the model training?

Optimization of w

This delivers the analytical solution:

$$\boldsymbol{w} = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

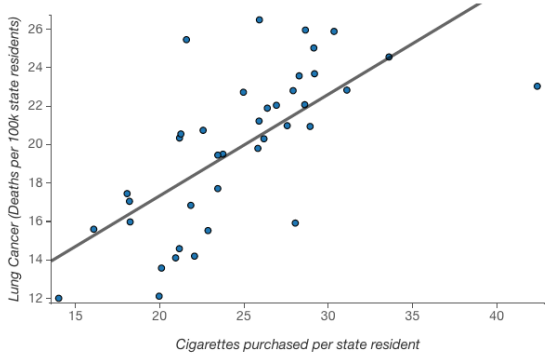
Question: What is the costly part of the model training?

$\rightarrow$ Calculation of inverse $(\boldsymbol{X}^\top \boldsymbol{X})^{-1} \in \mathbb{R}^{D \times D}$

- Gauss-Jordan elimination procedure: $\mathrm{O}\left(D^3\right)$
- Coppersmith-Winograd: $\mathrm{O}\left(D^{2.37\ldots}\right)$
- For large dimensions: stochastic gradient descent
  (nice: the error function is convex!)

Typical Use of Linear Regression

- To predict unknown values y_i for novel input variables x_i
- Visualization to understand a novel dataset: linear or non-linear relationships? Outliers? Distribution of residual errors?



- Estimate the influence of a single input variable or several variables (e.g. in medicine), i.e. estimating the strength of the **correlation** between x_i and y.
  BUT: does not allow for causal interpretation!

Questions to Answer for Yourself / Discuss with Friends

Repetition: What are the three assumptions required to have an analytical solution for the linear regression problem?

Questions to Answer for Yourself / Discuss with Friends

Repetition: What are the three assumptions required to have an analytical solution for the linear regression problem?

- Expected value of the residuals is 0: $\forall i : \mathrm{E}(\varepsilon_i) = 0$
- Residuals are uncorrelated and share the same variance: $\forall i : \mathrm{Var}(\varepsilon_i) = \sigma^2$
- Residuals follow a normal distribution: $\forall i : \varepsilon_i \sim \mathcal{N}(0, \sigma^2)$

Questions to Answer for Yourself / Discuss with Friends

Repetition: What are the three assumptions required to have an analytical solution for the linear regression problem?

- Expected value of the residuals is 0: $\forall i : \mathrm{E}(\varepsilon_i) = 0$
- Residuals are uncorrelated and share the same variance: $\forall i : \mathrm{Var}(\varepsilon_i) = \sigma^2$
- Residuals follow a normal distribution: $\forall i : \varepsilon_i \sim \mathcal{N}(0, \sigma^2)$

Ponder:

Can you think of counter-examples for these assumptions?

Outline

1 Linear Regression

2 Classification & Logistic Regression

3 Conclusion

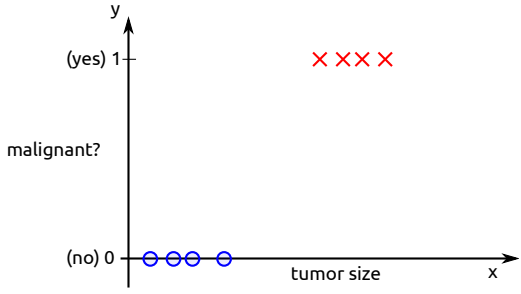


Simple Example: Tumor Classification

Is the tumor benign or malignant?

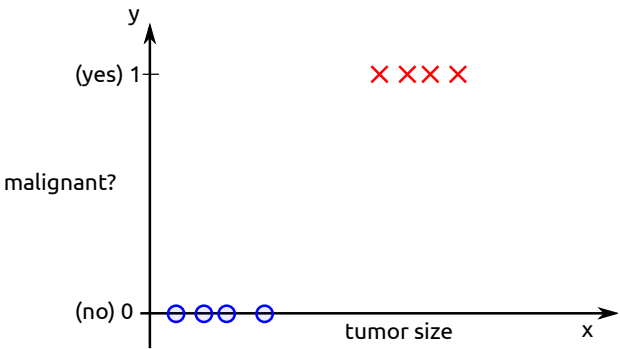
Labels $y \in \{0, 1\}$, with

- $y = 0$: ”Negative class” (e.g. benign, not malignant)
- $y = 1$: ”Positive class” (e.g. malignant)

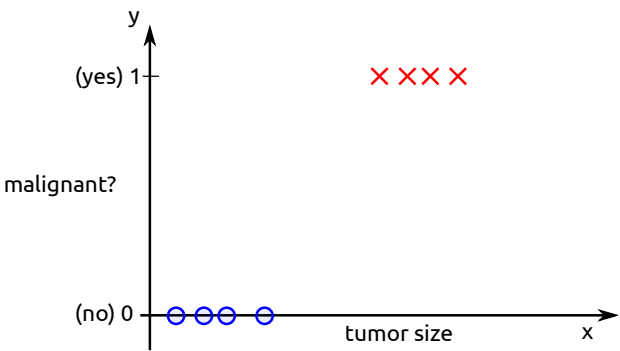


Try fitting a (augmented) model $h_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{w}^\top \boldsymbol{x}$ as in linear regression…

Oracle Solution for Tumor Classification



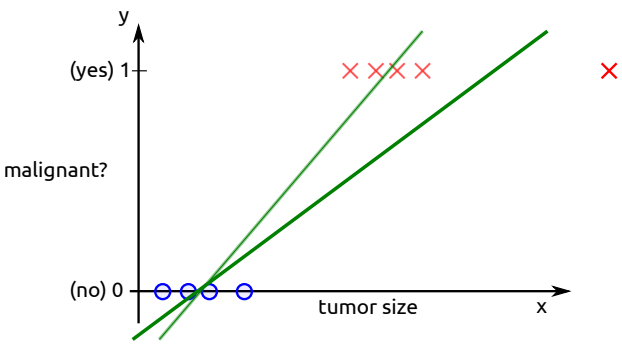
Oracle Solution for Tumor Classification



Oracle solution is:

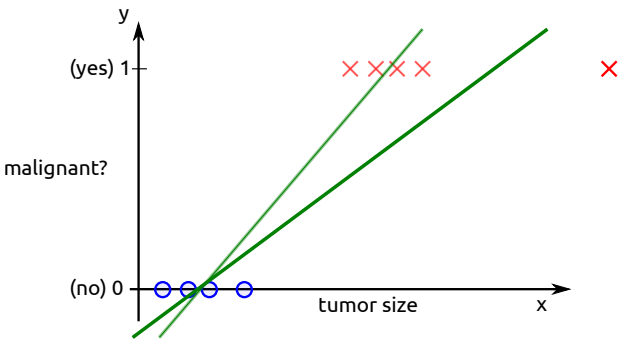
- If $h_{\boldsymbol{w}}(\boldsymbol{x}) > 0.5$,
  predict $y = 1$: (malignant)
- If $h_{\boldsymbol{w}}(\boldsymbol{x}) \leq 0.5$,
  predict $y = 0$: (not malignant)

Tumor Classification with Linear Regression



- For classification case we should only accept $y = 0$ or $y = 1$.
- However, linear regression model $h_{\boldsymbol{w}}(\boldsymbol{x})$ can generate $y > 1$ or $y < 0$.

Tumor Classification with Linear Regression



- For classification case we should only accept $y = 0$ or $y = 1$.
- However, linear regression model $h_{\boldsymbol{w}}(\boldsymbol{x})$ can generate $y > 1$ or $y < 0$.

Solution: **logistic regression**, which bounds the output to $0 \leq h_{\boldsymbol{w}}(\boldsymbol{x}) \leq 1$

Logistic Regression Model

We want: $0 \leq h_{\mathbf{w}}(\mathbf{x}) \leq 1$

Standard linear regression
(using the inner product) delivers:

$$h_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{w}^{\top}\boldsymbol{x}$$

Logistic Regression Model

We want: $0 \leq h_{\mathbf{w}}(\mathbf{x}) \leq 1$

Standard linear regression
(using the inner product) delivers:

$$h_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x}$$

A subtle change introduces non-linearity:

$$h_{\boldsymbol{w}}(\boldsymbol{x}) = g(\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x}),$$

with $g(z) = \dfrac{1}{1 + \mathrm{e}^{-z}}$.

Logistic Regression Model

We want: $0 \leq h_{\mathbf{w}}(\mathbf{x}) \leq 1$

Standard linear regression
(using the inner product) delivers:

$$h_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{w}^\mathsf{T}\boldsymbol{x}$$

A subtle change introduces non-linearity:

$$h_{\boldsymbol{w}}(\boldsymbol{x}) = g(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}),$$

with $g(z) = \dfrac{1}{1+\mathrm{e}^{-z}}$.

$$\Rightarrow h_{\boldsymbol{w}}(\boldsymbol{x}) = \frac{1}{1+\mathrm{e}^{-\boldsymbol{w}^\mathsf{T}\boldsymbol{x}}}$$

Logistic Regression Model

We want: $0 \leq h_{\mathbf{w}}(\mathbf{x}) \leq 1$

Standard linear regression
(using the inner product) delivers:

$$h_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x}$$

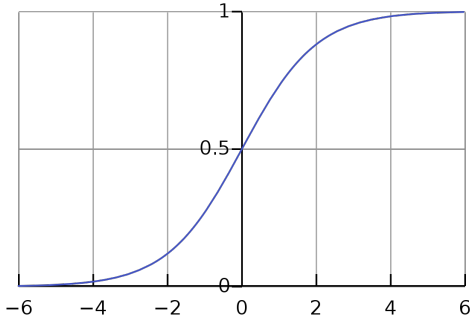
A subtle change introduces non-linearity:

$$h_{\boldsymbol{w}}(\boldsymbol{x}) = g(\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x}),$$

with $g(z) = \dfrac{1}{1+\mathrm{e}^{-z}}$.

$$\Rightarrow h_{\boldsymbol{w}}(\boldsymbol{x}) = \frac{1}{1+\mathrm{e}^{-\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x}}}$$

$g(z)$ is called **sigmoid function**
or **logistic function**



Interpretation of Model Output

$$h_{\boldsymbol{w}}(\boldsymbol{x}) \approx \text{estimated probability, that } y = 1$$

More formally, we work with a **hypothesis**:

Interpretation of Model Output

$$h_{\boldsymbol{w}}(\boldsymbol{x}) \approx \text{estimated probability, that } y = 1$$

More formally, we work with a **hypothesis**:

$$h_{\boldsymbol{w}}(\boldsymbol{x}) = \mathrm{P}(y = 1 \mid \boldsymbol{x};\ \boldsymbol{w})$$

"Probability that $y = 1$,
given that the input is $\boldsymbol{x}$ and
the model is parameterized by $\boldsymbol{w}$"

Interpretation of Model Output

$$h_{\boldsymbol{w}}(\boldsymbol{x}) \approx \text{estimated probability, that } y = 1$$

More formally, we work with a **hypothesis**:

$$h_{\boldsymbol{w}}(\boldsymbol{x}) = \mathrm{P}(y = 1 \mid \boldsymbol{x};\ \boldsymbol{w})$$

"Probability that $y = 1$, given that the input is $\boldsymbol{x}$ and the model is parameterized by $\boldsymbol{w}$"

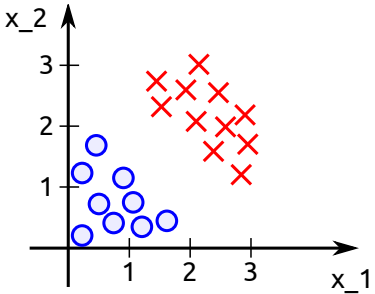
The actual labels are still discrete ($y = 0$ or $y = 1$), but the probabilities need to add to one:

$$\mathrm{P}(y = 0 \mid \boldsymbol{x};\ \boldsymbol{w}) + \mathrm{P}(y = 1 \mid \boldsymbol{x};\ \boldsymbol{w}) = 1$$

$$\mathrm{P}(y = 0 \mid \boldsymbol{x};\ \boldsymbol{w}) = 1 - \mathrm{P}(y = 1 \mid \boldsymbol{x};\ \boldsymbol{w})$$

Example in $\mathbb{R}^2$

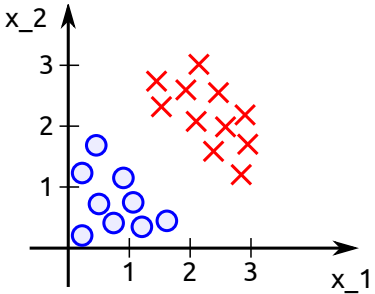
Where is the decision boundary?



$$h_{\boldsymbol{w}}(\boldsymbol{x}) = g(w_0 + w_1 x_1 + w_2 x_2)$$

Example in $\mathbb{R}^2$

Where is the decision boundary?



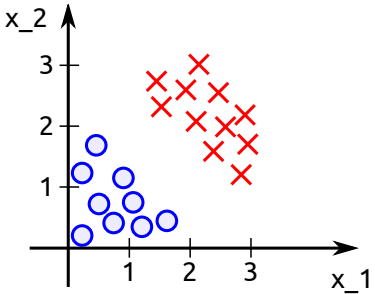
$$h_{\boldsymbol{w}}(\boldsymbol{x}) = g(w_0 + w_1 x_1 + w_2 x_2)$$

$$\boldsymbol{w} = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}$$

Predict "$y = 1$" if $-3 + x_1 + x_2 \geq 0$

Example in $\mathbb{R}^2$

Where is the decision boundary?



$$h_{\boldsymbol{w}}(\boldsymbol{x}) = g(w_0 + w_1 x_1 + w_2 x_2)$$

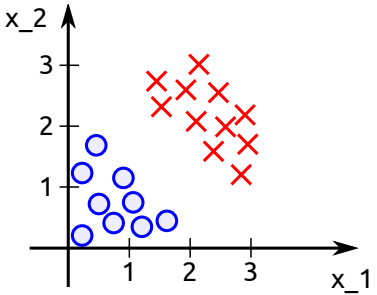
$$\boldsymbol{w} = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}$$

Predict "$y = 1$" if $-3 + x_1 + x_2 \geq 0$

$$x_1 + x_2 = 3 \iff h_{\boldsymbol{w}}(\boldsymbol{x}) = 0.5$$

Example in $\mathbb{R}^2$

Where is the decision boundary?



$$h_{\boldsymbol{w}}(\boldsymbol{x}) = g(w_0 + w_1 x_1 + w_2 x_2)$$

$$\boldsymbol{w} = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}$$

Predict "$y = 1$" if $-3 + x_1 + x_2 \geq 0$

$$x_1 + x_2 = 3 \iff h_{\boldsymbol{w}}(\boldsymbol{x}) = 0.5$$

Remark:

As in linear models, the use of additional dimensions and **basis functions** allows for non-linear decision boundaries!

Loss Function of Logistic Regression (I)

Proposed loss function (specifically adapted to logistic regression):

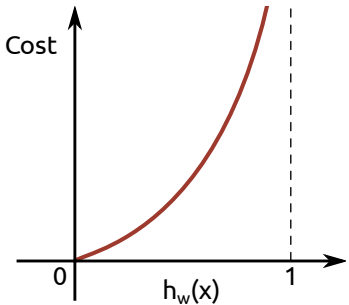
$$J(h_{\boldsymbol{w}}(y_i, \boldsymbol{x}_i)) = \begin{cases} -\log\left(1 - h_{\boldsymbol{w}}\left(\boldsymbol{x}_i\right)\right) & \text{for } y_i = 0 \\ -\log\left(h_{\boldsymbol{w}}\left(\boldsymbol{x}_i\right)\right) & \text{for } y_i = 1 \end{cases}$$

Loss Function of Logistic Regression (I)

Proposed loss function (specifically adapted to logistic regression):

$$J(h_{\boldsymbol{w}}(y_i, \boldsymbol{x}_i)) = \begin{cases} -\log\left(1 - h_{\boldsymbol{w}}\left(\boldsymbol{x}_i\right)\right) & \text{for } y_i = 0 \\ -\log\left(h_{\boldsymbol{w}}\left(\boldsymbol{x}_i\right)\right) & \text{for } y_i = 1 \end{cases}$$

Case: $y_i = 0$

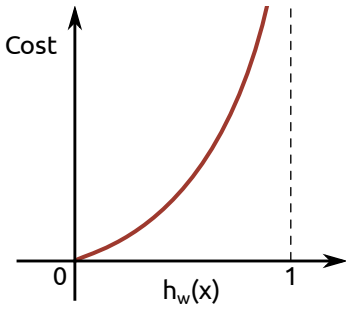


Loss Function of Logistic Regression (I)

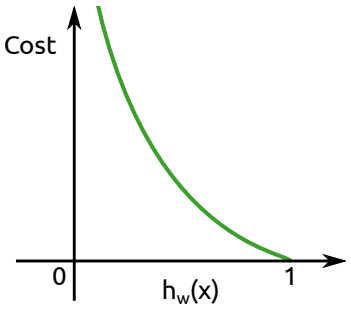
Proposed loss function (specifically adapted to logistic regression):

$$J(h_{\boldsymbol{w}}(y_i, \boldsymbol{x}_i)) = \begin{cases} -\log\left(1 - h_{\boldsymbol{w}}\left(\boldsymbol{x}_i\right)\right) & \text{for } y_i = 0 \\ -\log\left(h_{\boldsymbol{w}}\left(\boldsymbol{x}_i\right)\right) & \text{for } y_i = 1 \end{cases}$$

Case: $y_i = 0$



Case: $y_i = 1$



Loss Function for Logistic Regression (II)

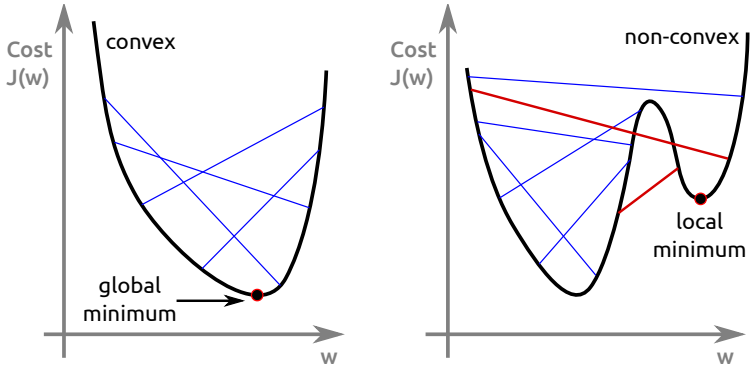
The case distinction can be avoided using this formulation
(check by setting $y_i = 0$ and $y_i = 1$):

$$J\left(h_{\boldsymbol{w}}\left(\boldsymbol{x}_i\right), y\right) = -\left(\left(1 - y_i\right) \log\left(1 - h_{\boldsymbol{w}}\left(\boldsymbol{x}_i\right)\right)\right) - y_i \log\left(h_{\boldsymbol{w}}\left(\boldsymbol{x}_i\right)\right)$$

- Nice property: J is **convex**.
- Not so nice property: There is no analytic solution (but gradient descent works well).

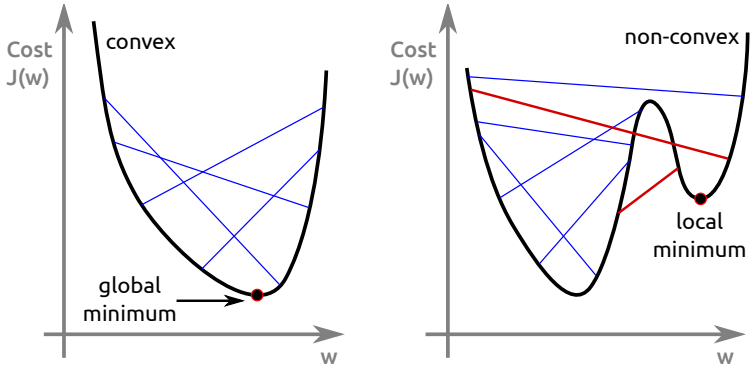
Convex Loss functions

- Loss for data point (x, y) depends on the choice of w: $J(h_w, y)$.



Convex Loss functions

- Loss for data point (x, y) depends on the choice of w: $J(h_w, y)$.
- Parameters w_i can be optimized easier, if the loss function is **convex** and (continuously) **differentiable**.



Iterative Optimization

Loss to minimize:

$$\begin{aligned}&\arg\min_{\boldsymbol{w}} \; J(\boldsymbol{w}) \\ &\qquad = \arg\min_{\boldsymbol{w}} \; \sum_{i=1}^{N} -y_i \log\left(h_{\boldsymbol{w}}\left(\boldsymbol{x}_i\right)\right) - \left(1-y_i\right) \log\left(1-h_{\boldsymbol{w}}\left(\boldsymbol{x}_i\right)\right)\end{aligned}$$

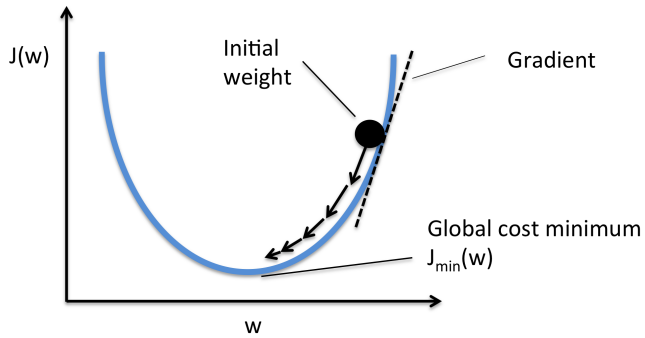
- Start with a guess for $\boldsymbol{w}$
- Move $\boldsymbol{w}$ towards the minimum of $J(\boldsymbol{w})$

How to train w for minimizing the loss?

Minimize $J(\boldsymbol{w})$ by updating $\boldsymbol{w}$

in the negative direction of $\frac{\partial J(\boldsymbol{w})}{\partial \boldsymbol{w}}$:

$$\boldsymbol{w}^{(\text{next})} \leftarrow \boldsymbol{w}^{(\text{prev})} - \eta \frac{\partial J(\boldsymbol{w})}{\partial \boldsymbol{w}^{(\text{prev})}}$$



Source: `http://rasbt.github.io/`

Gradient Descent

Find the optimal parameters $\overset{\star}{\boldsymbol{w}} \in \mathbb{R}^K$ that minimize an objective function $J(\boldsymbol{w})$, given data $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^N$, i.e.:

$$\overset{\star}{\boldsymbol{w}} := \arg\min_{\boldsymbol{w}} \; J(\mathcal{D}, \boldsymbol{w})$$

Algorithm 1: Gradient Descent Optimization

Input: Data $\mathcal{D}$, Learning rate $\eta \in \mathbb{R}_+$, Iterations $I \in \mathbb{N}_+$.

Ensure: $\boldsymbol{w} \in \mathbb{R}^K$

$w \sim \mathcal{N}(0, \sigma^2)$

for $1, \ldots, I$ **do**

  $\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \dfrac{\partial J(\mathcal{D}, \boldsymbol{w})}{\partial \boldsymbol{w}}$

return w

Stochastic Gradient Descent

Divide the dataset into R partitions (**mini-batches**) such that $\mathcal{D} = \bigcup_{r=1}^{R} \mathcal{D}_r$ and $\forall i \neq j,\ \mathcal{D}_i \cap \mathcal{D}_j = \emptyset$, yielding a decomposition of the loss:

$$J(\mathcal{D},\, \boldsymbol{w}) := \sum_{r=1}^{R} J(\mathcal{D}_r,\, \boldsymbol{w}) := \sum_{r=1}^{R} J_r$$

Algorithm 2: Stochastic Gradient Descent Optimization

Input: Data $\mathcal{D} = \bigcup_{r=1}^{R} \mathcal{D}_r$, Learning rate $\eta \in \mathbb{R}_+$, Iterations $I \in \mathbb{N}_+$.

Ensure: $\boldsymbol{w} \in \mathbb{R}^K$

```
w ~ N(0, σ²)
for 1, ..., I do
    foreach r ∈ {1, ..., R} in random order do
        w ← w − η ∂J_r/∂w
return w
```

Gradients

- For Linear Regression

$$\begin{aligned}\frac{\partial J(\boldsymbol{w})}{\partial \boldsymbol{w}} &= \frac{\partial \left[\sum_{i=1}^{N} \left(y_i - f\left(\boldsymbol{x}_i;\ \boldsymbol{w}\right)\right)^2\right]}{\partial \boldsymbol{w}} \\ &= \sum_{i=1}^{N} -2\left(y_i - f\left(\boldsymbol{x}_i;\ \boldsymbol{w}\right)\right)\boldsymbol{x}_i\end{aligned}$$

- For Logistic Regression

$$\begin{aligned}\frac{\partial J(\boldsymbol{w})}{\partial \boldsymbol{w}} &= \frac{\partial \left[\sum_{i=1}^{N} -y_i \log\left(h_{\boldsymbol{w}}\left(\boldsymbol{x}_i\right)\right) - \left(1 - y_i\right)\log\left(1 - h_{\boldsymbol{w}}\left(\boldsymbol{x}_i\right)\right)\right]}{\partial \boldsymbol{w}} \\ &= \sum_{i=1}^{N} -\left(y_i - h_{\boldsymbol{w}}\left(\boldsymbol{x}_i\right)\right)\boldsymbol{x}_i\end{aligned}$$

Questions to Answer for Yourself / Discuss with Friends

Repetition: Which function do we normally use to introduce a non-linearity to a classification model?

Repetition: What is the difference between Gradient Descent and Stochastic Gradient Descent?

Ponder: What advantages does SGD have over GD?

Questions to Answer for Yourself / Discuss with Friends

Repetition: Which function do we normally use to introduce a non-linearity to a classification model?

The **sigmoid** or **logistic** function, given by:

$$\sigma(z) = \frac{1}{1 + \mathrm{e}^{-z}}$$

Repetition: What is the difference between Gradient Descent and Stochastic Gradient Descent?

Ponder: What advantages does SGD have over GD?

Questions to Answer for Yourself / Discuss with Friends

Repetition: Which function do we normally use to introduce a non-linearity to a classification model?

The **sigmoid** or **logistic** function, given by:

$$\sigma(z) = \frac{1}{1 + \mathrm{e}^{-z}}$$

Repetition: What is the difference between Gradient Descent and Stochastic Gradient Descent?

In GD, we compute the loss and update the weights for the entire dataset in a single step.
In SGD, we split the dataset into **mini-batches** and update the model on them.

Ponder: What advantages does SGD have over GD?

Questions to Answer for Yourself / Discuss with Friends

Repetition: Which function do we normally use to introduce a non-linearity to a classification model?

The **sigmoid** or **logistic** function, given by:

$$\sigma(z) = \frac{1}{1 + \mathrm{e}^{-z}}$$

Repetition: What is the difference between Gradient Descent and Stochastic Gradient Descent?

In GD, we compute the loss and update the weights for the entire dataset in a single step.
In SGD, we split the dataset into **mini-batches** and update the model on them.

Ponder: What advantages does SGD have over GD?

- For large datasets, GD can be expensive, even infeasible due to memory constraints.
- SGD starts to improve with the first mini-batches. It doesn't need to wait for the entire dataset.

Outline

1 Linear Regression

2 Classification & Logistic Regression

3 Conclusion



Summary

In this lecture we have learned:

- The simplest linear methods for
  - Regression
  - Classification
- An analytical solution for the linear regression problem
- The use of basis functions to model non-linear data
- Gradient-based iterative solutions for the linear classification problem
  - Gradient Descent
  - Stochastic Gradient Descent

Method Recap

Method	Tasks		Complexity		Pros	Cons
	Class.	Regr.	Train / Fit	Inference		
K-Nearest Neighbors	B, MC	✓	—	$\mathrm{O}([k]\ n\ f)$	⊕ Simple and quick ⊕ Can handle noisy data	⊖ Expensive as attributes increase ⊖ Unweighted attributes
Naïve Bayes	B, MC	—	$\mathrm{O}(k\ n\ f)$	$\mathrm{O}(k\ f)$	⊕ Simple ⊕ Useful for large datasets ⊕ Requires little data ⊕ Probabilistic Predictions ⊕ Continuous and discrete data	⊖ Classes must be mutually exclusive ⊖ Assumes normally distr. attributes ⊖ Expects uncorrelated attributes
Linear Regression	—	✓	$\mathrm{O}\left(f^2(n+f)\right)$	$\mathrm{O}(f)$	⊕ Simple and interpretable ⊕ Analytic solution	⊖ Strong, unrealistic assumptions for residuals ⊖ Affected by outliers ⊖ Not scalable, unless done iteratively ⊖ Only for linear data, unless using basis functions
Linear Classification	B	—	$\mathrm{O}\left(f^2(n+f)\right)$	$\mathrm{O}(f)$	⊕ Simple ⊕ Useful for large datasets ⊕ Requires little data	⊖ Only for linearly separable data ⊖ Binary ⊖ Predictions are not probabilistic ⊖ Sensitive to outliers
Logistic Regression	B	—	$\mathrm{O}(n\ k\ f\ i)$	$\mathrm{O}(k\ f)$	⊕ Easy to implement ⊕ No assumptions about indep. variables ⊕ Probabilistic predictions	⊖ Only for linearly separable data ⊖ Binary, unless multinomial ⊖ Inaccurate for highly correlated inputs ⊖ Prone to overfitting